KLH UNIVERSITY

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING

PROJECT DOCUMENTATION

ARGUMENT PASSING

A Linux-Based Demonstration of Command-Line Argument Passing

Course Code	25CS2104E
Academic Year / Term	2026–27, Term-I
Section / Team	Section 6 / Team 2
Campus	KLH Bachupally Campus
Project Repository	github.com/yratnapriya05/2520030156_86_132_OS_PROJECT

TEAM MEMBERS

Student Name	Roll Number
Y. Ratna Priya	2520030156
J. Hasini	2520030086
N. Teja Sri	2520030132

TABLE OF CONTENTS

1. Abstract

2. Introduction

3. Problem Statement

4. Objectives

5. Scope of the Project

6. Existing and Proposed System

7. Feasibility Study

8. Operating System Concepts and Linux APIs

9. System Architecture

10. Proposed Methodology / Workflow

11. Tools and Technologies

12. Implementation Overview

13. Testing and Expected Results

14. Individual Team Contributions

15. Limitations

16. Future Enhancements

17. Conclusion

18. References

1. Abstract

Argument Passing is a system programming project that demonstrates how arguments can be passed from a parent process to a child process in a Linux environment. The project focuses on understanding how processes communicate and how command-line arguments are handled during program execution. It uses C programming and Linux process-management mechanisms to create a simple and practical demonstration of argument passing.

The system accepts arguments from the command line and passes or makes them available to the required process for execution. It demonstrates process creation, handling of process arguments, and execution of programs using Linux-based system programming concepts. The proposed solution is implemented in a Linux/Ubuntu environment using C programming and relevant POSIX/Linux APIs.

The expected outcome is a working application that clearly demonstrates how arguments are received, processed, and passed between processes, providing a practical understanding of operating-system and system-programming concepts.

2. Introduction

An operating system manages processes and provides mechanisms for starting and controlling programs. In Linux, a program can receive command-line arguments when it is started. These arguments allow a user or another program to provide input at execution time.

This project focuses on argument passing in a parent–child process scenario. A parent process can create a child process using fork(). The exec() family of functions can then be used to execute another program with a specified argument list. The project makes this flow easier to understand through a C-based demonstration in Linux/Ubuntu.

3. Problem Statement

The project aims to demonstrate how arguments are passed between processes in a Linux environment. It focuses on process creation, argument handling, and program execution using C programming and Linux system calls. The main learning problem addressed is understanding how command-line arguments are received by a program and made available to a child process or an executed program.

4. Objectives

- To understand the concept of command-line argument passing in Linux.
- To demonstrate how arguments are received and processed by a C program.
- To implement argument passing between processes using Linux system-programming mechanisms.
- To provide a simple practical demonstration of process creation and argument handling.

5. Scope of the Project

The scope of the project is limited to the following areas:

- Accepting command-line arguments in a C program.
- Demonstrating parent and child process creation in Linux.
- Using fork() to create a child process.

- Using the exec() family to execute a program with specified arguments.
- Displaying the arguments received and processed by the program.

The project is intended as an educational demonstration of Linux process and argument-handling concepts. Additional mechanisms such as pipes, shared memory, or signals are not included in the supplied project abstract.

6. Existing and Proposed System

6.1 Basic / Existing Approach

In a basic program, command-line arguments may be handled within a single program. Without a process-based demonstration, the movement of arguments from a parent process to a child process may remain difficult for learners to observe.

6.2 Proposed System

The proposed project demonstrates argument handling through a Linux-based C application. It accepts command-line arguments, creates a child process using fork(), and uses the exec() family to execute a program with the required arguments. The program output is used to show the arguments received.

Aspect	Basic / Existing Approach	Proposed Project
Process handling	Manual or limited process handling	Creates a child process using fork()
Argument handling	Argument flow may be less visible	Passes required arguments for execution
Program execution	Execution viewed as a single step	Uses exec() family with arguments
Learning outcome	Concept may remain theoretical	Practical Linux/C demonstration

7. Feasibility Study

7.1 Technical Feasibility

The project uses C programming, Linux/Ubuntu, the GCC compiler and POSIX/Linux process APIs. These tools support the planned process creation and program execution demonstration.

7.2 Economic Feasibility

The project can be developed using open-source software such as Linux/Ubuntu and GCC. No special hardware or hosting service is specified in the project proposal.

7.3 Operational Feasibility

The application is intended to be run from a Linux terminal using command-line input. The output displays the arguments received and processed by the program.

7.4 Schedule Feasibility

The project activities can be organized into design, implementation, testing, output verification and documentation within the semester timeline.

8. Operating System Concepts and Linux APIs

8.1 Process Creation

A process is a program in execution. Process creation is used to create and manage a new process for the demonstration.

8.2 Command-Line Arguments

Command-line arguments are values supplied when a program is started. In C, the main function can receive them through argc and argv.

```
int main(int argc, char *argv[])
```

Here, argc represents the number of command-line arguments available to the program, and argv is an array of argument strings.

8.3 fork()

fork() is a Linux/POSIX process-creation system call. It creates a child process from the calling process. The parent and child can then follow separate execution paths.

8.4 exec() Family

The exec() family of functions replaces the current process image with a new program. The functions provide ways to specify the program and its arguments.

8.5 Process Management

The project uses parent–child process execution to demonstrate how a process can create another process and arrange for a program to run with specified arguments.

8.6 POSIX APIs

POSIX APIs provide standard interfaces used for Linux/Unix process and system-programming operations.

9. System Architecture

The architecture below shows the high-level argument flow described in the project abstract.

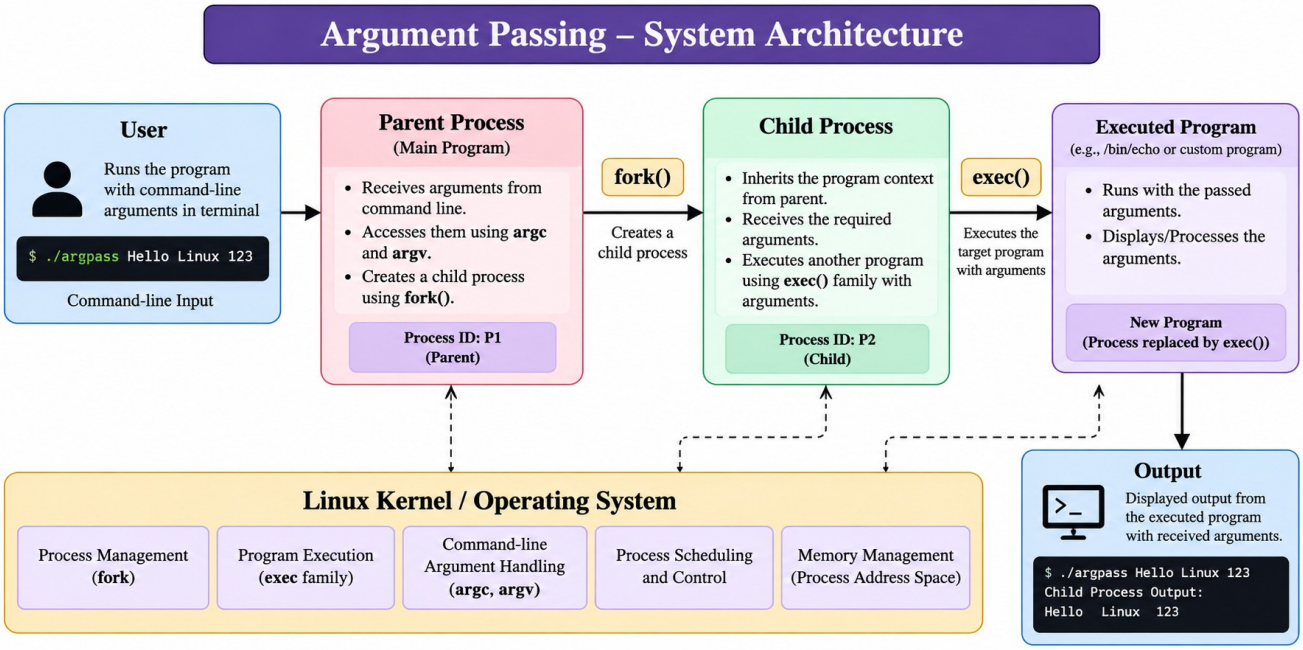


Figure 1: High-level system architecture for Argument Passing.

9.1 Architecture Components

- User: supplies command-line arguments in the terminal.
- Parent Process: receives arguments through argc and argv and initiates child-process creation.
- fork(): creates a child process.
- Child Process: follows the child execution path and prepares to run the target program.
- exec() family: executes the target program with the specified arguments.
- Output: the executed program displays or processes the received arguments.

10. Proposed Methodology / Workflow

1. User enters the required command-line arguments.
2. The parent C program receives the arguments through argc and argv.
3. The parent process calls fork() to create a child process.
4. The child process follows the required execution path.
5. The exec() family is used to execute the target program with the supplied arguments.
6. The executed program receives and processes the argument list.
7. The program displays the received or processed arguments.

The exact source-code sequence and output should be aligned with the final implementation maintained in the project repository.

11. Tools and Technologies

Tool / Platform / Software	Purpose
Linux / Ubuntu	Development and execution environment
C / C++	Project implementation (the abstract specifies C programming)
GCC Compiler	Compilation of C programs

POSIX APIs	Process and system-programming operations
GitHub	Source-code management and project repository

12. Implementation Overview

The project proposal specifies implementation in a Linux/Ubuntu environment using C programming. The program accepts arguments from the command line. A parent process can create a child process using the appropriate Linux process-management function, and the required arguments can be supplied to the child program during execution.

The main implementation concepts identified in the proposal are command-line argument handling, fork(), the exec() family, process management and POSIX APIs.

12.1 Illustrative Command

```
./argpass Hello Linux
```

Illustrative argument values: Hello and Linux. Replace the command and output below with the exact command and result produced by the final implementation.

12.2 Illustrative Output Format

```
Arguments received by the executed program:
1. Hello
2. Linux
```

This is an example format for explaining the concept, not a verified screenshot or recorded output from the project.

13. Testing and Expected Results

The project proposal states that the output will display the received and processed arguments. The following test cases are suggested to verify the behavior of the completed program.

Test Case	Input / Condition	Expected Observation
No arguments	Run without extra arguments	Program handles an empty argument list according to its implementation
Single argument	./argpass Hello	The executed program receives the supplied argument
Multiple arguments	./argpass Hello Linux	The executed program receives both argument values
Invalid / missing input	Use an invalid or incomplete input	Program behavior and error handling can be observed

Note: These are proposed test cases. Actual pass/fail status, screenshots, process IDs and measured results must be added after running the project.

14. Individual Team Contributions

Roll Number	Student Name	Responsibility
2520030156	Y. Ratna Priya	Project design, argument-passing implementation, documentation and GitHub management

2520030086	J. Hasini	Process creation and Linux system calls
2520030132	N. Teja Sri	Testing and output verification

15. Limitations

- The project proposal focuses on a basic demonstration of command-line argument passing.
- The exact behavior depends on the final C implementation and the selected exec() function.
- The supplied abstract does not specify pipes, shared memory, signals or other IPC mechanisms.
- Actual test results and execution screenshots must be included after implementation and verification.

16. Future Enhancements

- Add input validation and clearer error messages.
- Extend testing to special characters and larger argument lists.
- Improve the user interface or terminal output for easier demonstration.
- Add further process-management demonstrations if they are implemented and documented.

17. Conclusion

The Argument Passing project provides a practical way to understand command-line argument handling and parent–child process execution in Linux. By using C programming, fork(), the exec() family and POSIX/Linux APIs, the project demonstrates how a program can create a child process and execute another program with specified arguments. The project supports learning of fundamental operating-system and system-programming concepts.

18. References

8. Linux man-pages: fork(2). https://man7.org/linux/man-pages/man2/fork.2.html
9. Linux man-pages: exec(3). https://man7.org/linux/man-pages/man3/exec.3.html
10. Linux man-pages: execve(2). https://man7.org/linux/man-pages/man2/execve.2.html
11. GCC Documentation. https://gcc.gnu.org/onlinedocs/
12. Project Problem Statement Submission Form and Abstract supplied by the project team.
13. GitHub Repository: https://github.com/yratnapriya05/2520030156_86_132_OS_PROJECT